# PROJECT REPORT

**Abhineet.R**
25BCE10269

**VIT**yarthi Project

—

# SNAKE GAME

—

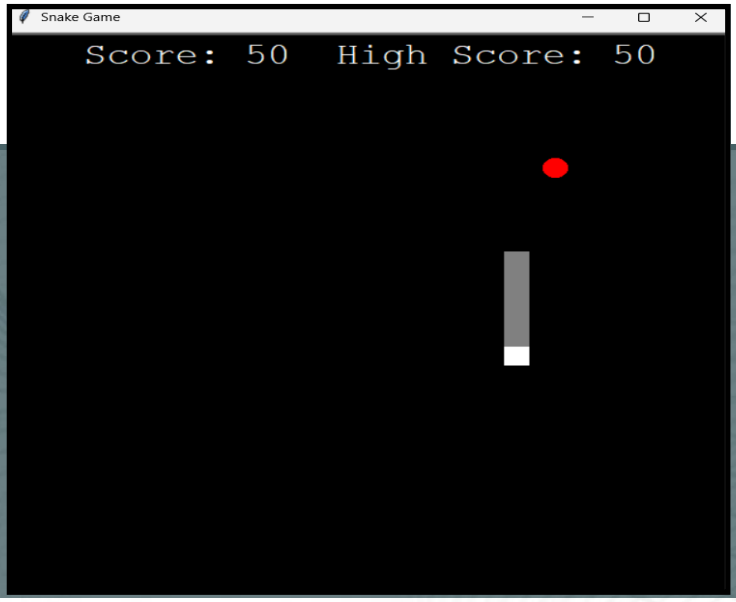
# INTRODUCTION

In the classic arcade game The Snake Game, the player takes control of a moving snake that gets longer every time it consumes food. The primary objective of the game is to continuously gather food items to raise the score while avoiding collisions with the snake's body or the walls.

Three common modules are used in this project to implement the Snake Game in Python:

## 1. The Turtle Module

The game's graphics are made using the turtle module. It offers basic commands for moving objects on the screen and creating shapes. Turtle graphics are used in this project to visualize the food, the game board, and the snake.



**2. The Time Module**

The game's speed is managed by the time module. The game operates smoothly and at a steady pace b y introducing tiny pauses between snake movement s.
This also makes it possible to change the difficulty b y speeding up or down.

**3. The Random Module**

The food appears in different places on the screen e ach time the snake consumes it because the random module creates random positions for the food.
This increases the game's unpredictability and mak es it more difficult and enjoyable.

Overall, by combining graphics, user input, and fun damental programming ideas like loops, conditions, and functions, this project shows how Python can b e used to create basic interactive games.

# PROBLEM STATEMENT

The game should include the following features:

- A snake that moves in four directions (up, down, left, right) based on keyboard inputs.
- Food that appears at random locations on the screen.
- The snake grows longer each time it eats food.
- A scoring system that updates the score and maintains a high score.
- Detection of collisions with the screen boundaries or the snake's own body, which should reset the game.
- Smooth graphics and movement using the turtle module and time delays.

# FUNCTIONAL REQUIREMENTS

**1. Game Window Setup**
- The system must create a 600×600 game window using the turtle module.
- The window must have a title, background color, and disabled auto-refresh for smooth animation.

**2. Snake Initialization**
- The system must create the snake's head at the center of the screen.
- The snake must initially remain stationary until a movement key is pressed.

**3. Food Generation**
- The system must generate food at a random position within the game boundary.
- The food must appear visually as a red circular object.

**4. Snake Movement**
- The system must allow the snake to move in four directions: up, down, left, and right.
- The snake must move continuously in the last given direction.
- Opposite-direction moves must be prevented (e.g., moving down immediately after moving up).

**5. Keyboard Controls**
- The system must respond to keyboard inputs:

    - **W** → Move Up
    - **S** → Move Down
    - **A** → Move Left
    - **D** → Move Right

**6. Growing the Snake**

- When the snake eats food:
    - A new body segment must be added.
    - Food must reappear at a new random location.

**7. Score Management**

- The system must maintain and display the player's score.
- The score must increase by 10 points every time food is eaten.
- The system must maintain and update a high score during the game session.

**8. Collision Detection**

The system must detect:

**a) Boundary Collision**

- If the snake hits the boundary, the game should reset:
    - Snake returns to the center.
    - Body segments are removed.
    - Score resets (high score remains).

**b) Self-Collision**

- If the snake's head touches any part of its body, the game must reset similarly.

**9. Body Segment Movement**

- The system must move the snake's body segments in sequence, following the head.
- Each segment must move to the position of the segment ahead of it.

**10. Game Loop**

- The game must run inside an infinite loop until the user manually closes the program.
- The screen must update continuously to reflect movement and changes.

**11. Game Speed Control**

- The system must use time delays to control the speed of the snake.
- Speed must remain consistent throughout gameplay.

# NON-FUNCTIONAL REQUIREMENTS

**1. Performance Requirements**

- The game must run smoothly with no lag during snake movement and screen updates.
- The system must refresh the display continuously using wn.update() to ensure real-time animation.
- The time delay (time.sleep(0.1)) must provide consistent game speed across different systems.

**2. Usability Requirements**

- The controls (W, A, S, D) must be easy to understand and responsive.
- The game interface must be simple, intuitive, and easy for beginners to play.
- Score and high score must be clearly visible at the top of the screen.

**3. Reliability Requirements**

- The game must run without crashes during continuous play.
- Collision detection must work accurately every time.
- The snake must always reset properly after crashing into the wall or itself.

**4. Maintainability Requirements**

- The code should be readable, modular, and easy to modify for future improvements.
- Functions like move(), go_up(), go_down(), etc., must be easy to update or expand.
- The program should be structured so additional features (levels, speed control, sound) can be added easily.

**5. Portability Requirements**

- The game must run on any system that supports Python and the turtle module (Windows, macOS, Linux).
- No external libraries should be required apart from built-in modules (turtle, time, random).

**6. Efficiency Requirements**

- The game must use minimal memory and CPU resources.
- The snake's movement and segment updates must be handled efficiently, even with a large number of segments.

**7. Visual Requirements**

- The game window must be visually clear with distinguishable colors (black background, white snake, red food).
- Text for score and high score must be readable.

**8. Safety Requirements**

- The game must not perform any harmful operations on the user's system.

- Only safe, built-in Python modules should be used.

**9. Responsiveness**

- The game must respond to key presses instantly.
- Any delay in movement direction must be minimal to support smooth gameplay.

# ALGORITHM

**Step 1: Start**
Begin the program execution.

**Step 2: Initialize Game Window**

- Create a screen using the turtle module.
- Set title, background colour, size, and disable auto-refresh using tracer(0).

**Step 3: Create the Snake Head**

- Create a turtle object for the snake head.
- Set shape, colour, speed, and initial direction ("stop").
- Position it at the centre of the screen.

**Step 4: Create Food**

- Create another turtle object for food.
- Set shape, colour, and place it at a starting position.

**Step 5: Initialize Variables**

- Create an empty list to store snake body segments.
- Set score = 0 and high score = 0.

**Step 6: Create Scoreboard**

- Create a turtle object for the scoreboard.
- Hide the turtle and display initial score and high score.

**Step 7: Define Direction Functions**
Create functions to change snake direction:

- Go up()
- Go down()
- Go left()
- Go right()

Ensure the snake cannot move in the opposite direction immediately.

**Step 8: Define Movement Function**

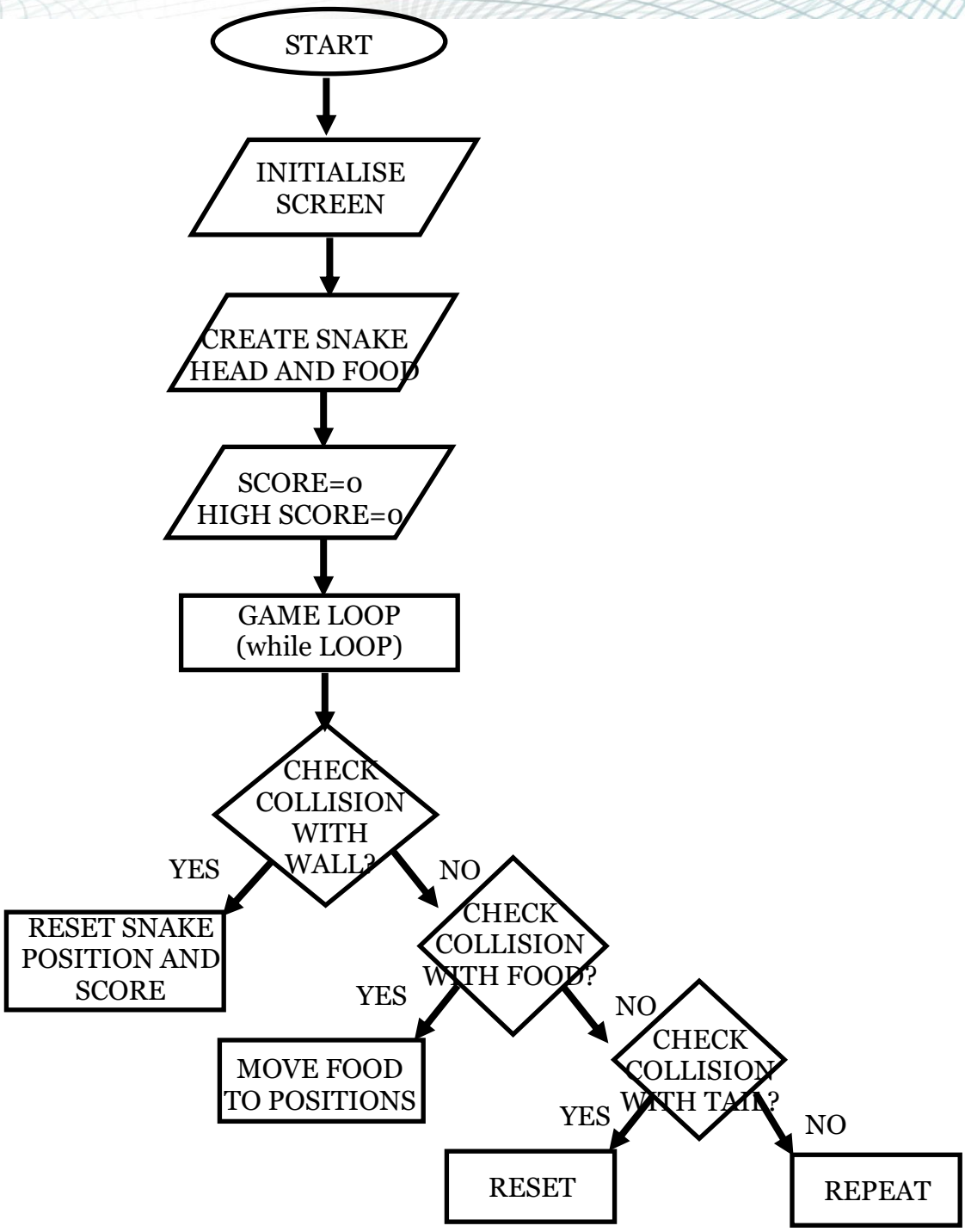
- Create a move() function.
- Update the snake's head position based on its direction:
    - If up → increase y-coordinate.
    - If down → decrease y-coordinate.
    - If left → decrease x-coordinate.
    - If right → increase x-coordinate.

**Step 9: Set Keyboard Controls**

- Listen for key presses (W, A, S, D).
- Bind them to the direction functions.

**Step 10: Start Main Game Loop (while True)**
Repeat continuously:

# THE FLOWCHART



# INPUT SCREENSHOTS

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
import turtle
import time
import random

# Set up the screen
wn = turtle.Screen()
wn.title("Snake Game")
wn.bgcolor("black")
wn.setup(width=600, height=600)
wn.tracer(0)  # Turns off screen updates for smoother animation

# Snake head
head = turtle.Turtle()
head.speed(0)
head.shape("square")
head.color("white")
head.penup()
head.goto(0, 0)
head.direction = "stop"

# Snake food
food = turtle.Turtle()
food.speed(0)
food.shape("circle")
food.color("red")
```

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
# Snake food
food = turtle.Turtle()
food.speed(0)
food.shape("circle")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []  # List to store snake body segments

# Scoreboard
pen = turtle.Turtle()
pen.speed(0)
pen.shape("square")
pen.color("white")
pen.penup()
pen.hideturtle()
pen.goto(0, 260)
pen.write("Score: 0  High Score: 0", align="center", font=("Courier", 24, "normal"))

score = 0
high_score = 0
```

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
# Functions for changing direction
def go_up():
    if head.direction != "down":
        head.direction = "up"

def go_down():
    if head.direction != "up":
        head.direction = "down"

def go_left():
    if head.direction != "right":
        head.direction = "left"

def go_right():
    if head.direction != "left":
        head.direction = "right"
```

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
# Function for moving the snake
def move():
    if head.direction == "up":
        y = head.ycor()
        head.sety(y + 20)

    if head.direction == "down":
        y = head.ycor()
        head.sety(y - 20)

    if head.direction == "left":
        x = head.xcor()
        head.setx(x - 20)

    if head.direction == "right":
        x = head.xcor()
        head.setx(x + 20)

# Keyboard bindings
wn.listen()
wn.onkeypress(go_up, "w")
wn.onkeypress(go_down, "s")
wn.onkeypress(go_left, "a")
wn.onkeypress(go_right, "d")
```

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
# Main game loop
while True:
    wn.update()

    # Check for collision with border
    if head.xcor() > 290 or head.xcor() < -290 or head.ycor() > 290 or head.ycor() < -290:
        time.sleep(1)
        head.goto(0, 0)
        head.direction = "stop"

        # Hide the segments
        for segment in segments:
            segment.goto(1000, 1000)  # Move off-screen
        segments.clear()

        # Reset score
        score = 0
        pen.clear()
        pen.write(f"Score: {score}  High Score: {high_score}", align="center", font=("Courier", 24, "normal"))
```

snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
    # Check for collision with food
    if head.distance(food) < 20:
        # Move the food to a random location
        x = random.randint(-280, 280)
        y = random.randint(-280, 280)
        food.goto(x, y)

        # Add a new segment to the snake
        new_segment = turtle.Turtle()
        new_segment.speed(0)
        new_segment.shape("square")
        new_segment.color("grey")
        new_segment.penup()
        segments.append(new_segment)

        # Increase score
        score += 10

        if score > high_score:
            high_score = score

        pen.clear()
        pen.write(f"Score: {score}  High Score: {high_score}", align="center", font=("Courier", 24, "normal"))
```

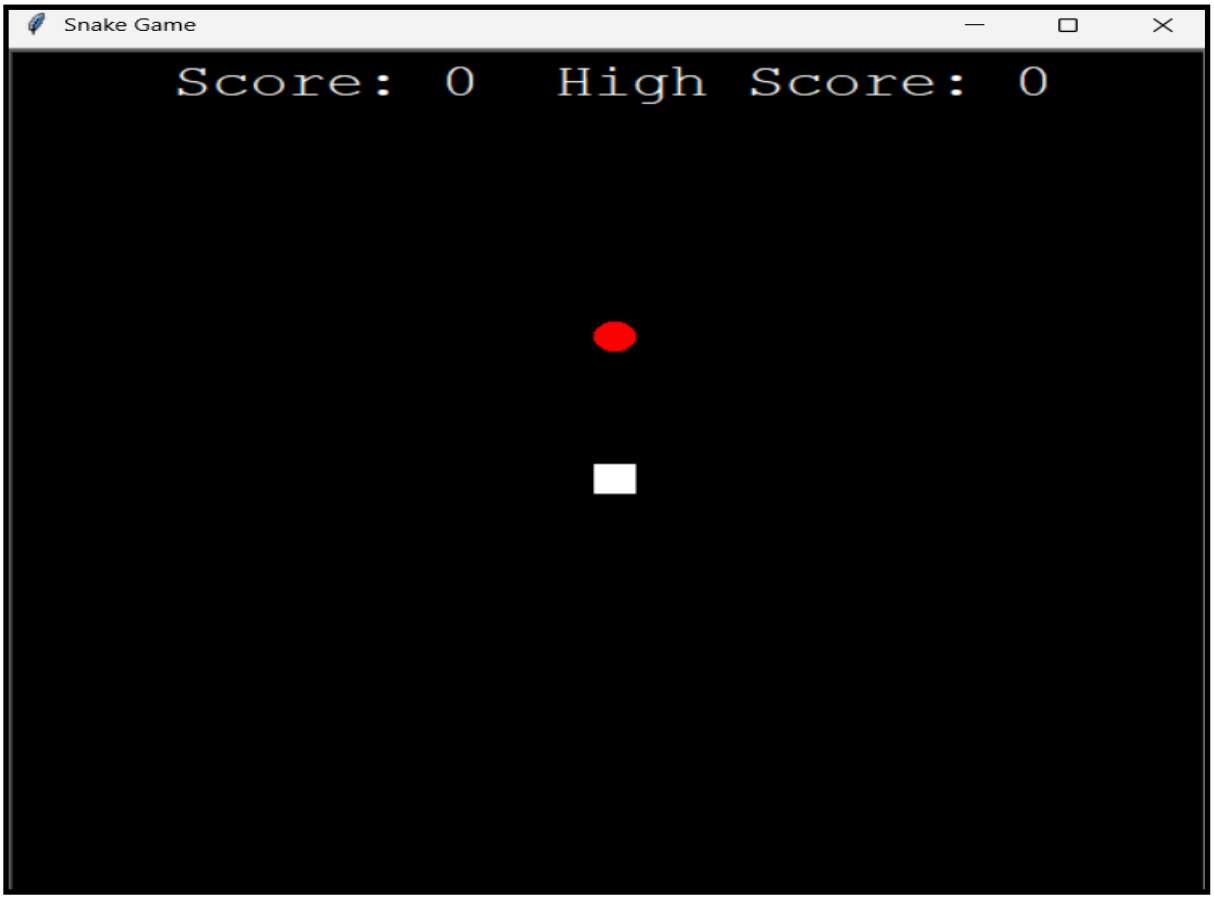
snake game.py - C:\Users\Abhineet\AppData\Local\Programs\Python\Python39\snake game.py (3.9.4)

File Edit Format Run Options Window Help

```python
    # Move the end segments first in reverse order
    for index in range(len(segments) - 1, 0, -1):
        x = segments[index - 1].xcor()
        y = segments[index - 1].ycor()
        segments[index].goto(x, y)

    # Move segment 0 to where the head is
    if len(segments) > 0:
        x = head.xcor()
        y = head.ycor()
        segments[0].goto(x, y)

    move()

    # Check for head collision with body segments
    for segment in segments:
        if segment.distance(head) < 20:
            time.sleep(1)
            head.goto(0, 0)
            head.direction = "stop"

            # Hide the segments
            for segment in segments:
                segment.goto(1000, 1000)
            segments.clear()
```

```python
        # Reset score
        score = 0
        pen.clear()
        pen.write(f"Score: {score}  High Score: {high_score}", align="center", font=("Courier", 24, "normal"))

time.sleep(0.1)  # Controls the game speed
```

# OUTPUT SCREENSHOTS



# PROJECT SUMMARY

## SNAKE GAME

## 1. Introduction

Games have always been an engaging method to learn programming concepts. This project focuses on developing a **Snake Game** using Python. The game uses the built-in **turtle**, **time**, and **random** modules to create graphics, control movement, manage speed, and generate random food positions. The project demonstrates essential programming concepts such as loops, conditionals, event handling, collision detection, and modular programming.

---

## 2. Objectives of the Project

The main goals of this project are:

- To develop a fully functional Snake Game using Python.
- To understand the use of turtle graphics for creating simple 2D animations.
- To explore random module usage for generating unpredictable food positions.
- To implement game logic including movement, scoring, and collision detection.
- To enhance logical and problem-solving skills while developing a game.

---

## 3. Required Modules

| Module | Purpose |
|---|---|
| **turtle** | Used to draw the snake, food, scoreboard, and screen interface. |
| **time** | Controls game speed using delays. |
| **random** | Generates random coordinates for food placement. |

All three modules are built-in, so no installation is required.

---

## 4. System Requirements

- **Software:** Python 3.x
- **Hardware:** Any basic laptop/desktop
- **OS:** Windows / macOS / Linux
- **Editor:** IDLE, VS Code, PyCharm, or any code editor that supports Python

---

## 5. Project Description

The Snake Game consists of:

1. **A snake (player-controlled)** that moves across the screen using arrow keys.
2. **Food** placed at random positions.
3. **Scoreboard** to track the score.
4. **Game Over logic** when the snake hits the wall or itself.

The player controls the movement direction, and the snake grows longer as it eats food. The difficulty increases naturally as the snake becomes longer, making the game more challenging.

---

## 6. Flow of the Program

1. **Initialize** the screen using turtle.
2. **Create** the snake head.
3. **Create** the food object.
4. **Create** the scoreboard.
5. **Define movement functions** (up, down, left, right).
6. **Run the main loop**:
    - Update the screen
    - Move the snake
    - Detect collision with food
    - Detect collision with wall/self
    - Update score
    - Adjust the speed using time.sleep()
7. **Display Game Over message** when collision occurs.

**********

# OOPS!
# CRASHED

# TESTING APPROACH

Testing the Snake Game ensures that all components—game mechanics, graphics, and logic—work correctly and smoothly. The following structured testing approach covers different types of testing.

---

## 1. Functional Testing

Verify whether each feature of the game works as intended.

### Test Scenarios

1. **Game Window Setup**
   - The game window opens with the correct title, background color, and size.
   - The snake and food are visible after game start.
2. **Snake Movement**
   - Snake moves continuously in the chosen direction.
   - Arrow keys correctly change the snake's direction.
   - Snake should not move backward (prevent opposite-direction movement).
3. **Food Collision**
   - When snake head touches food, the food relocates.
   - Snake body increases by 1 segment.
   - Score updates correctly.
4. **Boundary Collision**
   - Snake dies when it touches the boundary of the window.
   - Game resets (head returns to center, segments removed, score resets).
5. **Body Collision**
   - Snake dies when its head touches any segment of its own body.
   - Game resets properly.
6. **Scoreboard**
   - Score displays correctly.
   - Highest score is updated properly.

---

## 2. Performance Testing

Check how well the game behaves under stress.

**Key Checks**

- Game continues smoothly even as snake length becomes long.
- No lag when many segments are present.

- Delay value used in game loop maintains constant speed.

---

## 3. Input Testing

Test all keyboard controls.

- Up arrow → Snake moves upward.
- Down arrow → Snake moves downward.
- Left arrow → Snake moves left.
- Right arrow → Snake moves right.
- No input should crash the game.
- Repeated key presses should not break movement.

---

## 4. Boundary Testing

Check extreme values and limits.

- Test collision when snake head is exactly on the boundary edge.
- Test food spawning at edges.
- Test game reset if food or snake spawns near boundary.

---

## 5. Error Handling Testing

Ensure the game doesn't crash unexpectedly.

- Rapid direction changes should not cause snake to move backward.
- Game should reset properly without throwing errors.
- No infinite loops or turtle errors during gameplay.

---

## 6. Usability Testing

Check how user-friendly the game is.

- Are controls easy to use?
- Are score and messages visible?
- Is the snake movement smooth and intuitive?
- Are restart behaviors clear to the player?

---

## 7. Compatibility Testing

Ensure the game runs on different environments.

- Test on different screen sizes/resolutions.
- Test on Windows, Mac, and Linux (if possible).
- Runs in different Python versions (3.8–3.12).

---

## 8. Regression Testing

After updating code or fixing bugs:

- Re-test all basic functionalities.
- Ensure no new bugs appear (especially in movement and collision logic).

# POSSIBLE CHALLENGES FACED DURING THE CODE

## Game logic challenges

- Handling movement smoothly on a grid (consistent step size, frame rate, and direction changes without “skipping” or jitter).
- Correctly growing the snake after eating food while keeping the body list in the right order (head at index 0, tail at the end). A small mistake here often causes weird body jumps or no growth at all.
- Detecting collisions accurately:
  - Snake with walls (off-screen positions).
  - Snake with itself (head position matching any body segment except the head).
  - Snake with food (coordinate equality; must match grid size).

## Input and controls

- Preventing instant 180-degree turns (e.g., going right and immediately pressing left) which should be disallowed or handled carefully.
- Handling keyboard input and the event loop properly (not mixing pygame.event.get() and pygame.key.get_pressed() incorrectly, avoiding missed or repeated keypresses).

## Code design and scaling

- Avoiding overly long while loops or “god functions” that handle everything; instead separating logic into functions or classes (Snake, Food, Game).
- Adding features (levels, score display, sound, menus) without breaking existing code, which requires some initial planning and modular design.

# Future Enhancements for the Snake Game

### 1. Add Levels / Difficulty Modes

- Introduce **Easy, Medium, Hard** levels.
- Increase snake speed or obstacle count with each level.

### 2. Add Sound Effects

- Play sounds when:
  - Snake eats food
  - Snake hits the wall
  - Game over

  (Using winsound module on Windows)

### 3. Obstacles in the Game Area

- Add random blocks/walls inside the game.
- Snake must avoid hitting them.

### 4. High Score Saving Feature

- Save high score to a **file (highscore.txt)**.
- Load it when the game starts.

### 5. Multiple Food Types

- Normal food → +10 points
- Special food → +25 points (appears occasionally)
- Poison food → -10 points

### 6. Snake Skins / Color Themes

- Allow user to select:
  - Snake color
  - Background color
  - Food color

### 7. Smooth Movement Instead of 90° Turns

- Use continuous movement for a more modern feel.
- Move snake on a grid but allow smoother transitions.

**8. Boundary Wrap Feature**

- Instead of dying on wall collision, snake appears at opposite side (like classic Nokia snake).

**9. Game Menu Screen**

Add a main menu with:

- Start Game
- Help / Instructions
- Quit

**10. Pause & Resume Function**

- Press "P" to pause
- Press "R" to resume

**11. Mobile Controls**

- Add buttons on screen for:
    - Up, Down, Left, Right
- Useful for touch-screen devices.

**12. Two-Player Mode**

- Player 1: WASD
- Player 2: Arrow keys
- Two snakes compete for food.

**13. Timer Mode**

- Player must score maximum points in 60 seconds.

**14. Improving Graphics**

- Replace simple square snake blocks with **pixel graphics or sprites** for a modern look.

**15. Add Animation Effects**

- Food blinking
- Smooth snake growth
- Fade-out game over animation

# CONCLUSION

The Snake Game developed using Python's **turtle**, **time**, and **random** modules successfully demonstrates the fundamental concepts of programming, including loops, functions, event handling, and object movement on a graphical interface. The project replicates the classic snake gameplay by allowing the snake to move continuously, consume food, grow in size, and end the game upon collision with the wall or its own body.

Through this project, important concepts such as **game logic**, **collision detection**, **score management**, and **screen refresh control** were implemented effectively. The program also highlights how simple Python libraries can be combined to create an interactive and visually engaging game.

Overall, the Snake Game project provided hands-on experience with Python graphics and game development, strengthened logical thinking skills, and served as a foundation for understanding how real-time games function. With additional features such as sound, levels, obstacles, and improved UI, the game can be further enhanced into a more complex and polished version.

**Done By:**

**ABHINEET.R**
**Reg No: 25BCE10269**
**Applc No: 2025073153**
**Branch: CSE Core**